

INFORMATICS INSTITUTE OF TECHNOLOGY

INFORMATICS INSTITUTE OF TECHNOLOGY

In Collaboration with

UNIVERSITY OF WESTMINSTER

Module code: 6NTCM009W Internet of Things (IoT)

Assessment Title: Coursework – An IoT prototype

Student name: Malindu Wijayarathna

Student ID: 20200646 / w1870586

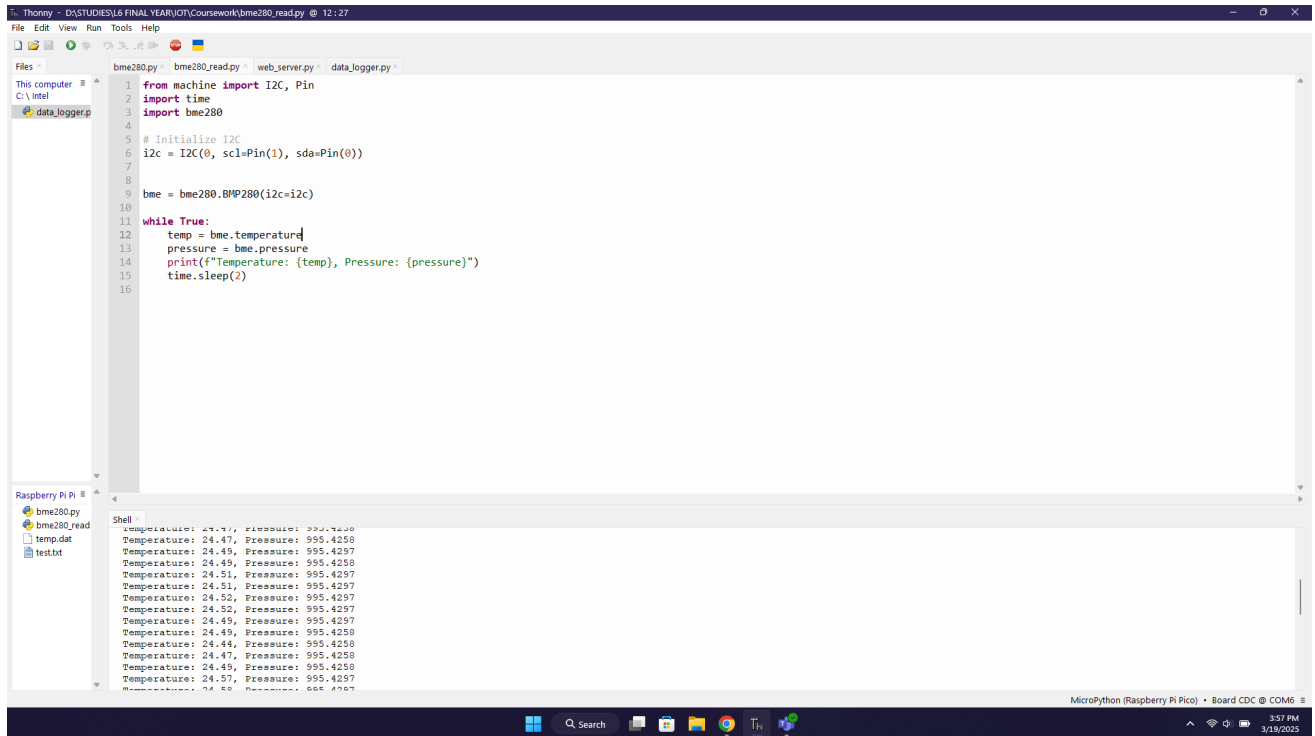
Table of contents

Contents

Table of contents	ii
A. DEVELOPMENT Part:	1
1. Getting temperature and humidity.....	1
2. Web server.....	2
3. Data logger	4
B. TESTING Part:	6
1. Raspberry Pico W setup	6
2. Data visualization.....	8
3. Handling connection issues.....	9
4. Expanding to an IoT air pollution monitoring system	9
Appendix	11
Appendix A – bme280_read	11
Appendix B – web_server	11
Appendix C – data_logger.....	16
Appendix D – Apps Script Code.....	18

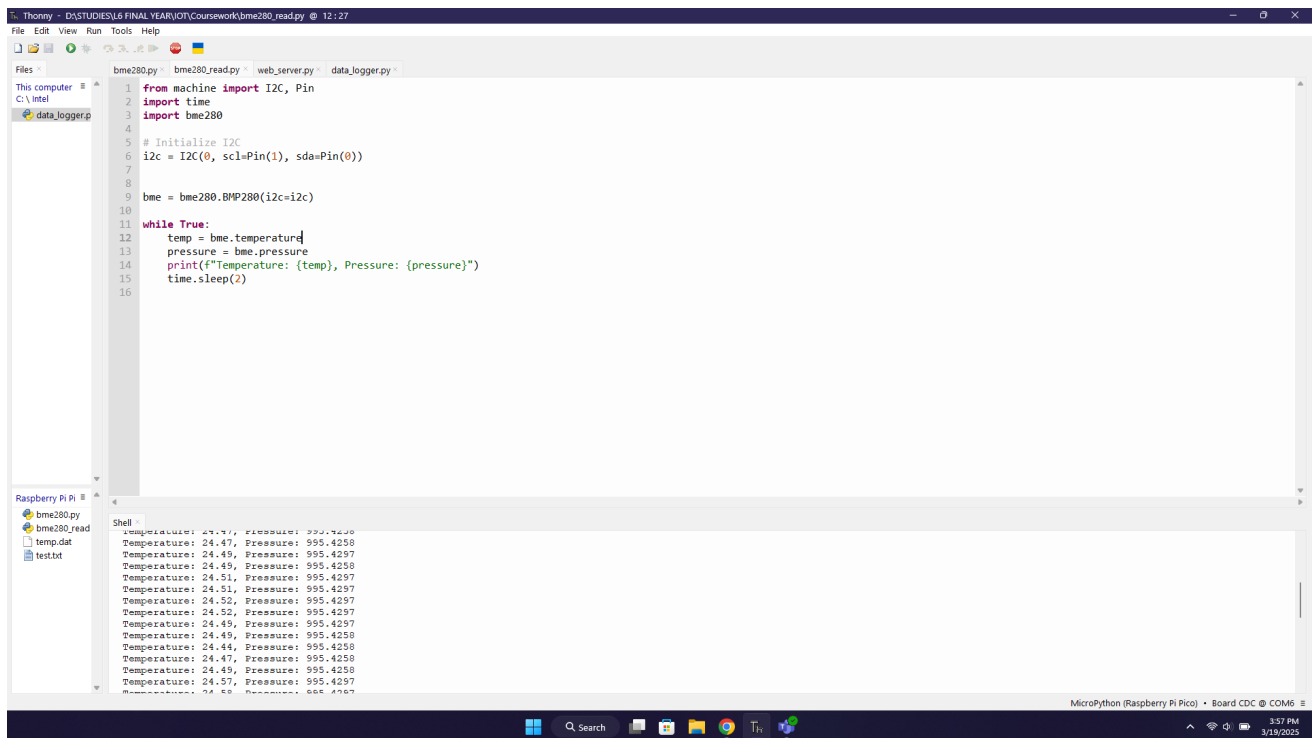
A. DEVELOPMENT Part:

1. Getting temperature and humidity



```
1 from machine import I2C, Pin
2 import time
3 import bme280
4
5 # Initialize I2C
6 i2c = I2C(0, scl=Pin(1), sda=Pin(0))
7
8
9 bme = bme280.BME280(i2c=i2c)
10
11 while True:
12     temp = bme.temperature
13     pressure = bme.pressure
14     print(f"Temperature: {temp}, Pressure: {pressure}")
15     time.sleep(2)
16
```

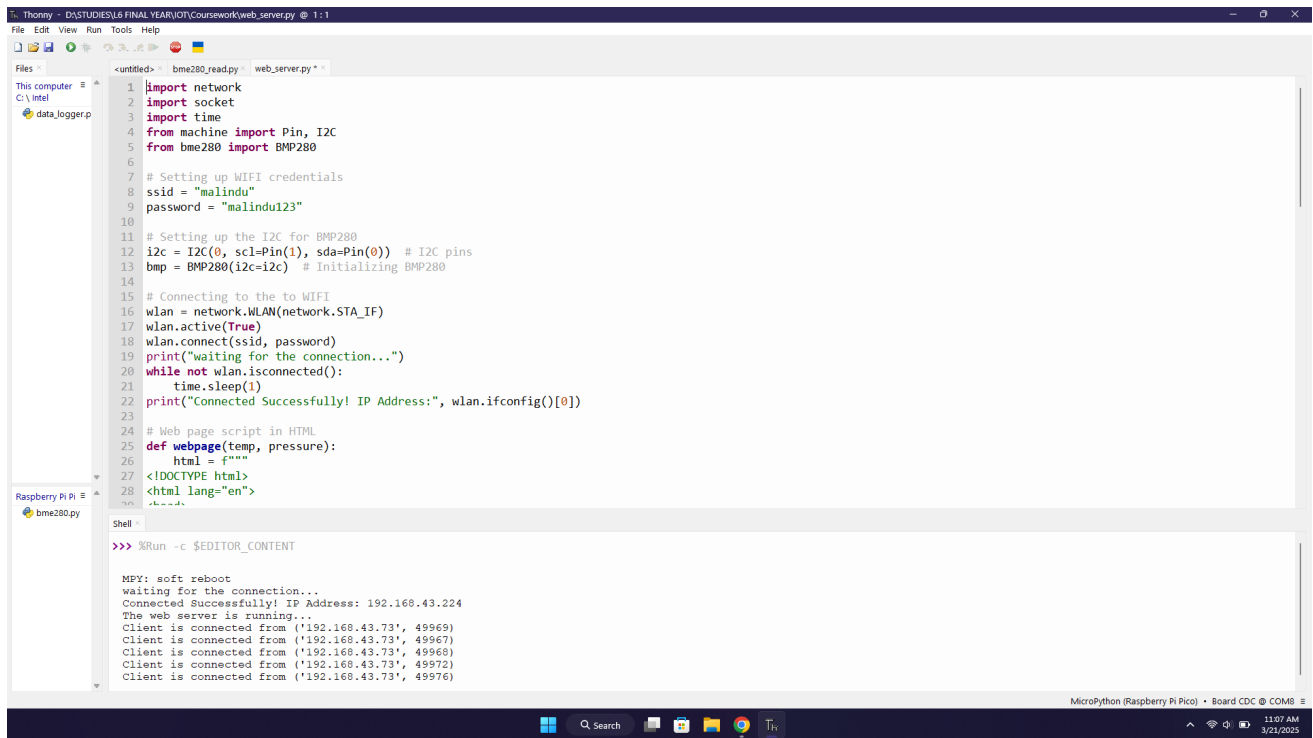
Temperature: 24.77, Pressure: 995.4250
Temperature: 24.47, Pressure: 995.4250
Temperature: 24.49, Pressure: 995.4297
Temperature: 24.49, Pressure: 995.4258
Temperature: 24.51, Pressure: 995.4297
Temperature: 24.51, Pressure: 995.4297
Temperature: 24.52, Pressure: 995.4297
Temperature: 24.52, Pressure: 995.4297
Temperature: 24.49, Pressure: 995.4297
Temperature: 24.49, Pressure: 995.4258
Temperature: 24.44, Pressure: 995.4258
Temperature: 24.47, Pressure: 995.4258
Temperature: 24.49, Pressure: 995.4258
Temperature: 24.57, Pressure: 995.4297
Temperature: 24.58, Pressure: 995.4297



```
1 from machine import I2C, Pin
2 import time
3 import bme280
4
5 # Initialize I2C
6 i2c = I2C(0, scl=Pin(1), sda=Pin(0))
7
8
9 bme = bme280.BME280(i2c=i2c)
10
11 while True:
12     temp = bme.temperature
13     pressure = bme.pressure
14     print(f"Temperature: {temp}, Pressure: {pressure}")
15     time.sleep(2)
16
```

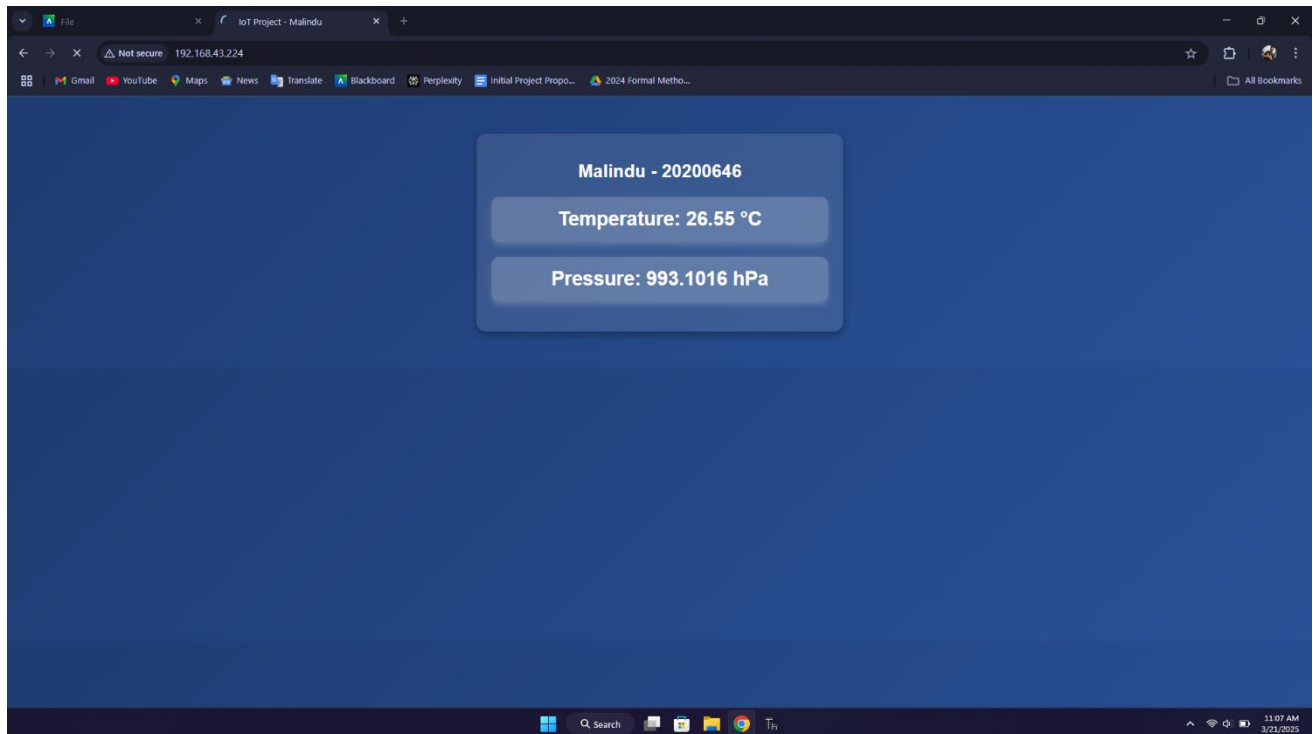
Temperature: 24.77, Pressure: 995.4250
Temperature: 24.47, Pressure: 995.4250
Temperature: 24.49, Pressure: 995.4297
Temperature: 24.49, Pressure: 995.4258
Temperature: 24.51, Pressure: 995.4297
Temperature: 24.51, Pressure: 995.4297
Temperature: 24.52, Pressure: 995.4297
Temperature: 24.52, Pressure: 995.4297
Temperature: 24.49, Pressure: 995.4297
Temperature: 24.49, Pressure: 995.4258
Temperature: 24.44, Pressure: 995.4258
Temperature: 24.47, Pressure: 995.4258
Temperature: 24.49, Pressure: 995.4258
Temperature: 24.57, Pressure: 995.4297
Temperature: 24.58, Pressure: 995.4297

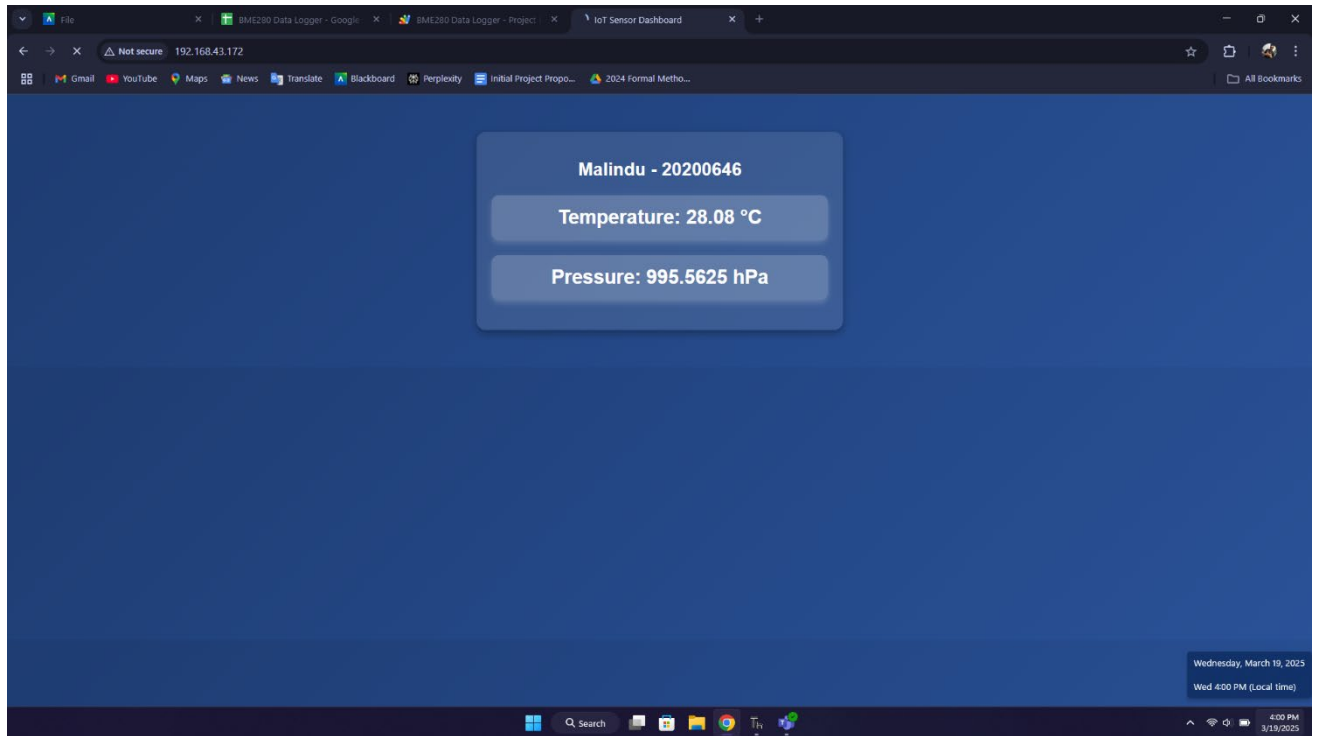
2. Web server



The screenshot shows the Thonny IDE with a file named `web_server.py` open. The code is a Python script for a web server on a Raspberry Pi Pico, using the MicroPython environment. It imports necessary modules like `network`, `socket`, and `time`. It sets up a WLAN connection with the SSID "malindu" and password "malindu123". It also initializes a BMP280 sensor for temperature and pressure readings. The script defines a `webpage` function that returns an HTML response with the current sensor data. The main loop connects to the WLAN and serves the webpage to any connected clients.

```
1 import network
2 import socket
3 import time
4 from machine import Pin, I2C
5 from bme280 import BMP280
6
7 # Setting up WIFI credentials
8 ssid = "malindu"
9 password = "malindu123"
10
11 # Setting up the I2C for BMP280
12 i2c = I2C(0, scl=Pin(1), sda=Pin(0)) # I2C pins
13 bmp = BMP280(i2c=i2c) # Initializing BMP280
14
15 # Connecting to the WIFI
16 wlan = network.WLAN(network.STA_IF)
17 wlan.active(True)
18 wlan.connect(ssid, password)
19 print("waiting for the connection...")
20 while not wlan.isconnected():
21     time.sleep(1)
22 print("Connected Successfully! IP Address:", wlan.ifconfig()[0])
23
24 # Web page script in HTML
25 def webpage(temp, pressure):
26     html = f"""
27     <!DOCTYPE html>
28     <html lang="en">
29     </html>
30 """
31
32 >>> %Run -c $EDITOR_CONTENT
33
34 MPY: soft reboot
35 waiting for the connection...
36 Connected Successfully! IP Address: 192.168.43.224
37 The web server is running...
38 Client is connected from ('192.168.43.73', 49969)
39 Client is connected from ('192.168.43.73', 49967)
40 Client is connected from ('192.168.43.73', 49968)
41 Client is connected from ('192.168.43.73', 49972)
42 Client is connected from ('192.168.43.73', 49976)
```





3. Data logger

The image shows a Thonny IDE window with a Python script for a BME280 data logger. The script imports necessary modules, sets up WiFi credentials, and connects to a Google Sheet to log temperature and pressure data. Below the script, the Shell window shows the execution output, including the IP address and the data being logged.

```
1 import network
2 import urequests
3 import time
4 from machine import Pin, I2C
5 from bme280 import BMP280
6
7 # Setting up WIFI credentials
8 ssid = "malindu"
9 password = "malindu123"
10
11 # Web App URL for the Google sheet
12 SCRIPT_URL = "https://script.google.com/macros/s/AKfycbz-1stKumV6znXh8Y6e0fC1G5IrHq3bHgEziP-Stg_lageGwpRc8UpaLp1q90yS0aYK/exec"
13
14 # Setting up the I2C for BMP280
15 i2c = I2C(0, scl=Pin(1), sda=Pin(0))
16 bmp = BMP280(i2c=i2c)
17
18 # Connecting to the WIFI
19 wlan = network.WLAN(network.STA_IF)
20 wlan.active(True)
```

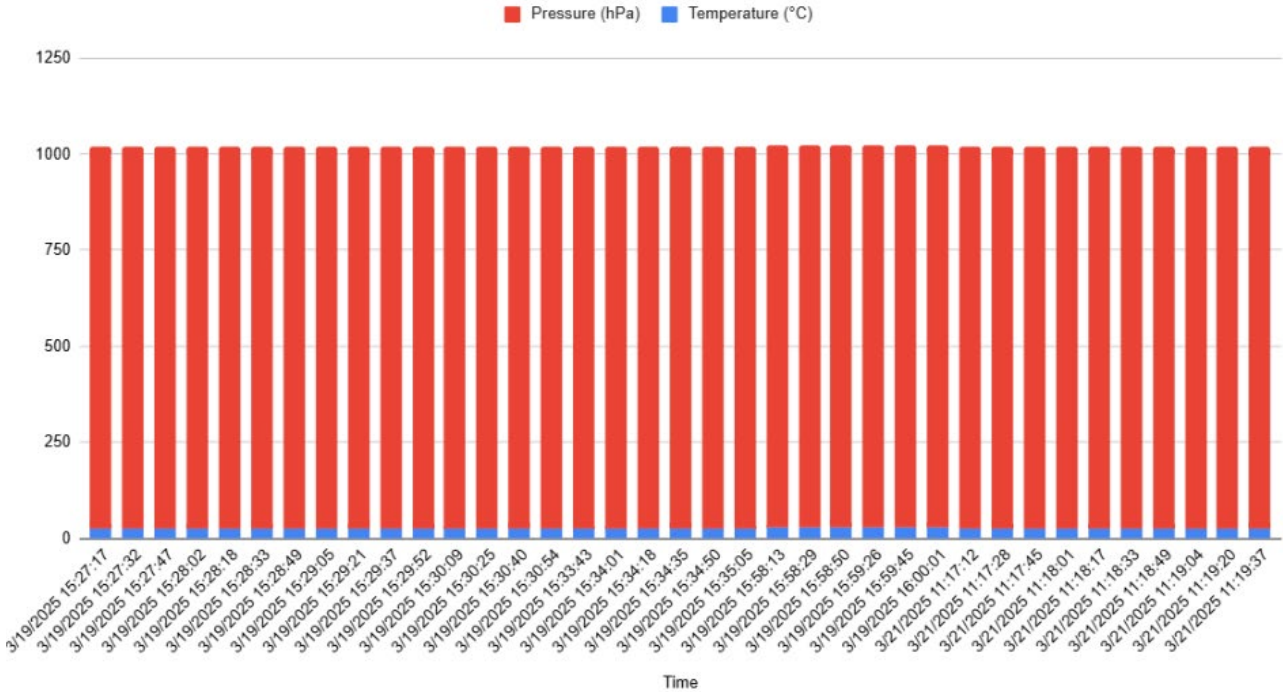
The Shell window shows the following output:

```
MPY: soft reboot
waiting for the connection...
Connected Successfully! IP Address: 192.168.43.224
Response: <!DOCTYPE html><html lang="si"><head><meta name="description" content="BME280 Data Logger"></head></html>
Logged: 26.65 °C, 993.11 hPa
Response: <!DOCTYPE html><html lang="si"><head><meta name="description" content="BME280 Data Logger"></head></html>
Logged: 26.57 °C, 993.1 hPa
Response: <!DOCTYPE html><html lang="si"><head><meta name="description" content="BME280 Data Logger"></head></html>
Logged: 26.56 °C, 993.1 hPa
Response: <!DOCTYPE html><html lang="si"><head><meta name="description" content="BME280 Data Logger"></head></html>
Logged: 26.5 °C, 993.1 hPa
Response: <!DOCTYPE html><html lang="si"><head><meta name="description" content="BME280 Data Logger"></head></html>
Logged: 26.51 °C, 993.1 hPa
Response: <!DOCTYPE html><html lang="si"><head><meta name="description" content="BME280 Data Logger"></head></html>
Logged: 26.49 °C, 993.1 hPa
Response: <!DOCTYPE html><html lang="si"><head><meta name="description" content="BME280 Data Logger"></head></html>
Logged: 26.46 °C, 993.1 hPa
Response: <!DOCTYPE html><html lang="si"><head><meta name="description" content="BME280 Data Logger"></head></html>
Logged: 26.47 °C, 993.1 hPa
Response: <!DOCTYPE html><html lang="si"><head><meta name="description" content="BME280 Data Logger"></head></html>
```

The Google Sheet titled "BME280 Data Logger" contains the following data:

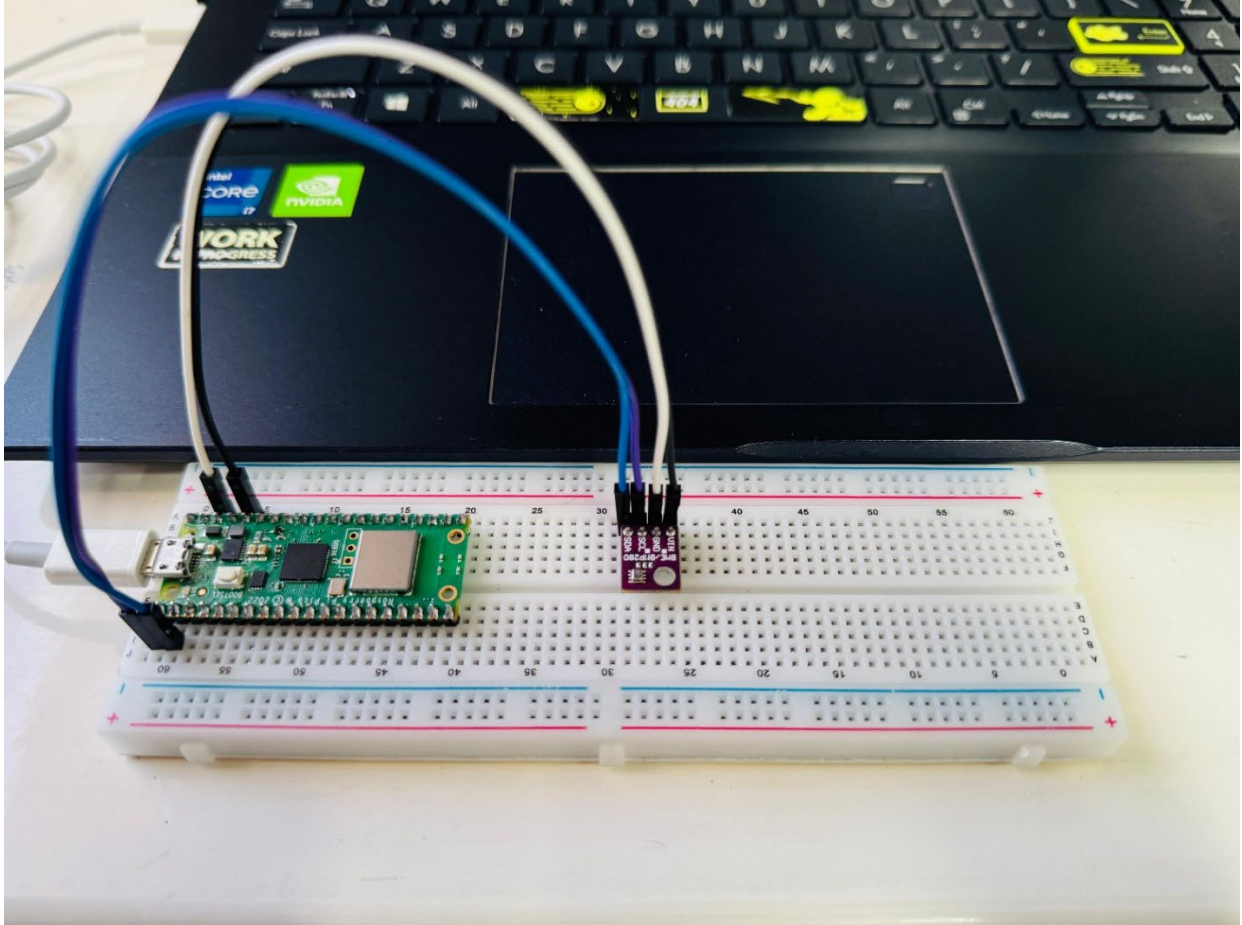
A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T
1	Time	Temperature (°C)	Pressure (hPa)																
2	3/19/2025 15:27:17	24.64	995.43		152														
3	3/19/2025 15:27:32	24.61	995.43																
4	3/19/2025 15:27:47	24.58	995.43																
5	3/19/2025 15:28:02	24.55	995.43																
6	3/19/2025 15:28:18	24.55	995.43																
7	3/19/2025 15:28:33	24.45	995.43																
8	3/19/2025 15:28:49	24.45	995.43																
9	3/19/2025 15:29:05	24.46	995.43																
10	3/19/2025 15:29:21	24.43	995.43																
11	3/19/2025 15:29:37	24.47	995.43																
12	3/19/2025 15:29:52	24.47	995.43																
13	3/19/2025 15:30:09	24.48	995.43																
14	3/19/2025 15:30:25	24.44	995.43																
15	3/19/2025 15:30:40	24.47	995.43																
16	3/19/2025 15:30:54	24.45	995.43																
17	3/19/2025 15:33:43	24.57	995.43																
18	3/19/2025 15:34:01	24.62	995.43																
19	3/19/2025 15:34:18	24.58	995.43																
20	3/19/2025 15:34:35	24.63	995.43																
21	3/19/2025 15:34:50	24.59	995.43																
22	3/19/2025 15:35:05	24.53	995.43																
23	3/19/2025 15:58:13	27.07	995.52																
24	3/19/2025 15:58:29	27.41	995.54																
25	3/19/2025 15:58:50	27.39	995.54																
26	3/19/2025 15:59:26	27.47	995.54																
27	3/19/2025 15:59:45	28.03	995.56																
28	3/19/2025 16:00:01	28.02	995.56																
29	3/21/2025 11:17:12	26.85	993.11																
30	3/21/2025 11:17:28	26.57	993.1																
31	3/21/2025 11:17:45	26.56	993.1																
32	3/21/2025 11:18:01	26.5	993.1																
33	3/21/2025 11:18:17	26.51	993.1																
34	3/21/2025 11:18:33	26.49	993.1																
35	3/21/2025 11:18:49	26.46	993.1																
36	3/21/2025 11:19:04	26.47	993.1																
37	3/21/2025 11:19:20	26.45	993.1																
38	3/21/2025 11:19:37	26.54	993.1																
39	3/21/2025 11:19:53	26.46	993.1																
40	3/21/2025 11:19:58	26.46	993.1																

Temperature (°C) and Pressure (hPa)



B. TESTING Part:

1. Raspberry Pico W setup



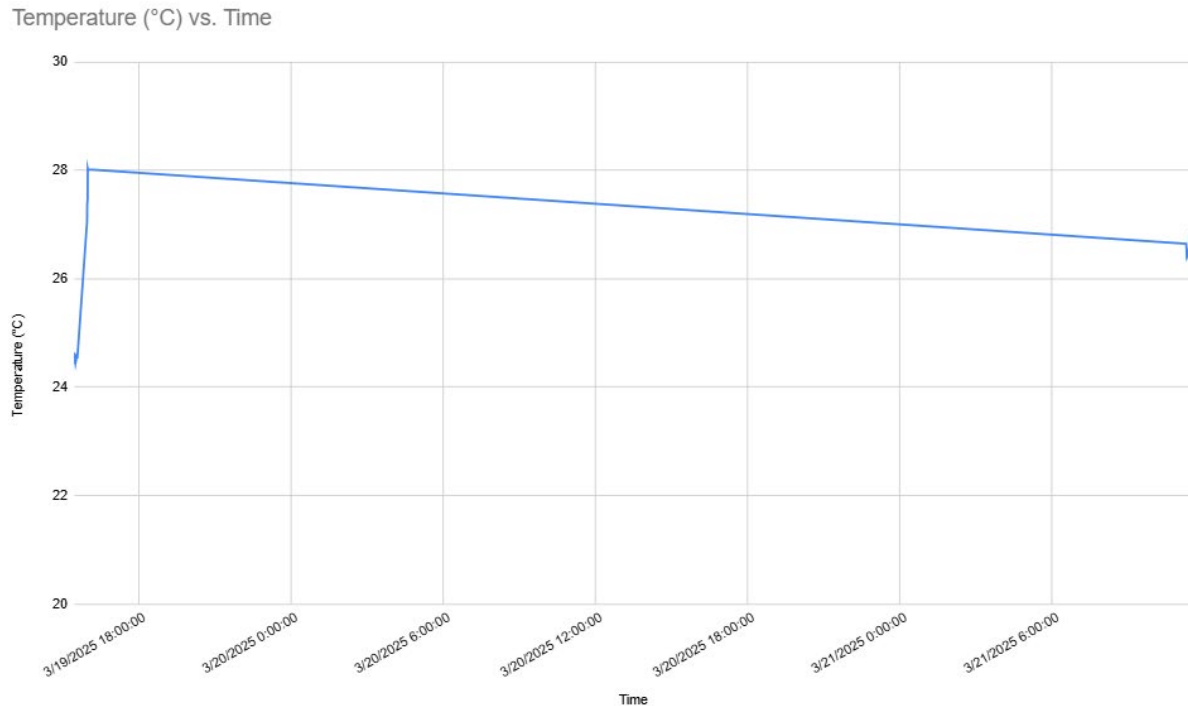
The prototype was tested to verify its functionality between the Raspberry Pico w and the BME280 sensor under following steps.

1. Setup and power connection was established by connecting the Pico W with BME280 sensor using four jumper wires on the breadboard. The SDA was connected to Pin 1, SCL connected with Pin 2, GND connected with Pin 38 and finally the VIN connected with Pin 36 and the system was powered via a USB cable connected to the PC.
2. Wi-Fi connection was established by programming Pico W to connect to a Wi-Fi network with providing the relevant SSID and password and was successfully connected and verified through the connection status that output.

3. Sensor data readings that were logged to the console were cross checked with an external thermometer to validate the outputs.
4. The web server was established to display real time sensor data through a web server and the results were checked by accessing the web page through PC and checked whether it updates timely.
5. Data logging was established by sending the sensor data via HTTP requests to the google sheet app script and the google sheet was checked to verify that it updates timely with correct time stamps and sensor data.

Test Case	Expected Result	Actual Result	Pass/Fail
Setting up the setup	Setting up was done successfully	Setting up was done successfully	Pass
Pico W connects with Wi-Fi	Successfully connects and outputs the IP address	Connects and displays IP address	Pass
Sensor detects both temperature and Pressure	Sensor identifies correctly	Correct values are identified	Pass
Web server displays data	Webpage shows sensor data	Sensor data are shown and updates timely	Pass
Data logged to google sheet	Google sheet updated with sensor readings	Sensor readings were updated with timestamp	Pass

2. Data visualization



The above graphs depict the temperature readings over time, which is plotted from the data that is collected in different environmental conditions.

1. The X axis shows the timestamp
2. The Y axis shows the temperature

Through this graph it can be clearly identify the temperature differences that occurred during various environments in different dates which are subjected to indoor and outdoor conditions. It can be clearly identified the spike in temperature in indoor condition and outdoor condition on 19/03/2025 and a slight decrease in temperature identified in 20/03/2025 and a spike and a drop identified in 21/03/2025.

Through these visualizations it helps to analyze and identify the temperature variations that occur over time, and it helps to easily compare the differential environmental conditions readings with others and its useful to identify sudden temperature or pressure drops or spikes that occur in real time when monitoring them in real time. Therefore, these visualizations are important, and they provide insights for weather-based decision making.

3. Handling connection issues

To handle the connection issues the script should be modified to detect the connection lost and should be able to reconnect automatically while preventing the data loss during that time by retrying with the failed requests that were received.

```
def connect_wifi():
    wlan = network.WLAN(network.STA_IF)
    wlan.active(True)
    wlan.connect(ssid, password)

    print("Connecting to Wi-Fi...")
    retries = 0
    while not wlan.isconnected():
        time.sleep(2)
        retries += 1
        if retries > 10:
            print("Wi-Fi connection failed. Restarting...")
            machine.reset() # Restarting if WIFI fails
    print("Connected! IP Address:", wlan.ifconfig()[0])

# Sending data with retries
def send_to_sheets(temp, pressure):
    """ Send sensor data to Google Sheets with error handling """
    attempt = 0
    while attempt < 5: # Retrying upto 5 attempts
        try:
            url = f"{SCRIPT_URL}?temp={temp}&pressure={pressure}"
            response = urequests.get(url)
            print("Response:", response.text)
            response.close()
            return
        except Exception as e:
            print("Error sending data:", e)
            attempt += 1
            time.sleep(5) # Waiting before trying
    print("Failed to send data after multiple attempts.")

# Checking WIFI connection and reconnecting if lost
def check_wifi():
    wlan = network.WLAN(network.STA_IF)
    if not wlan.isconnected():
        print("Wi-Fi lost! Reconnecting...")
        connect_wifi()

# Connecting to WIFI before starting
connect_wifi()

while True:
    check_wifi()
    temp = bmp.temperature
    pressure = bmp.pressure
    send_to_sheets(temp, pressure)
    print(f"Logged: {temp} °C, {pressure} hPa")
    time.sleep(10)
```

A Wi-Fi reconnection method was introduced here to continuously check the connection and to attempt reconnection and a method to retry before sending data to the google sheet. Through this if there is a connection lost, the reading is buffered and sent when the connection is established again, and it was successfully achieved.

4. Expanding to an IoT air pollution monitoring system

To expand this prototype into an IoT air pollution monitoring system needs additional further improvements to be made. As this current prototype uses the BMP280 sensor which detects the temperature and pressure for air pollution monitoring it requires additional sensors for it.

1. Gas sensor modules like MQ-135 are needed to identify separately the harmful gases like carbon dioxide and carbon monoxide which are in the air.
2. Particulate matter sensors like PMS5003 are needed to detect PM1, PM2.5, PM10 particulates which are present in air for air quality monitoring.
3. Humidity sensors like DHT22 are needed for monitoring humidity of the air as the humidity directly affects the air pollution dispersion, it needs to be monitored.

Through adding these sensors and using more visualizations, this current prototype can be enhanced and expanded further to air pollution monitoring system that provides real time air quality insights.

Appendix

Appendix A – bme280_read

```
from machine import I2C, Pin

import time

import bme280

# Initialize I2C

i2c = I2C(0, scl=Pin(1), sda=Pin(0))

# Initialize the BMP280 sensor

bme = bme280.BMP280(i2c=i2c)

while True:

    temp = bme.temperature # Read temperature

    pressure = bme.pressure # Read Pressure

    print(f'Temperature: {temp}, Pressure: {pressure}')

    time.sleep(2) # Waiting time
```

Appendix B – web_server

```
import network

import socket

import time

from machine import Pin, I2C

from bme280 import BMP280
```

```

# Setting up WIFI credentials

ssid = "malindu"

password = "malindu123"


# Setting up the I2C for BMP280

i2c = I2C(0, scl=Pin(1), sda=Pin(0)) # I2C pins

bmp = BMP280(i2c=i2c) # Initializing BMP280


# Connecting to the to WIFI

wlan = network.WLAN(network.STA_IF)

wlan.active(True)

wlan.connect(ssid, password)

print("waiting for the connection...")

while not wlan.isconnected():

    time.sleep(1)

print("Connected Successfully! IP Address:", wlan.ifconfig()[0])


# Web page script in HTML

def webpage(temp, pressure):

    html = f"""

    <!DOCTYPE html>

    <html lang="en">

    <head>

```

```
<meta charset="UTF-8">

<meta name="viewport" content="width=device-width, initial-scale=1.0">

<meta http-equiv="refresh" content="5">

<title>IoT Project - Malindu</title>

<style>

    body {{

        font-family: 'Arial', sans-serif;

        text-align: center;

        background: linear-gradient(135deg, #1e3c72, #2a5298);

        color: white;

        margin: 0;

        padding: 0;

    }}

    .container {{

        max-width: 450px;

        margin: 50px auto;

        padding: 20px;

        background: rgba(255, 255, 255, 0.1);

        border-radius: 12px;

        box-shadow: 0 4px 10px rgba(0,0,0,0.3);

    }}

    h1 {{

        font-size: 24px;

        margin-bottom: 10px;
```

```

    }}

    .sensor-box {{
        font-size: 26px;
        margin: 20px 0;
        padding: 15px;
        background: rgba(255, 255, 255, 0.2);
        border-radius: 10px;
        box-shadow: 0px 4px 8px rgba(255,255,255,0.2);
        font-weight: bold;
    }}

    .footer {{
        font-size: 14px;
        margin-top: 15px;
        opacity: 0.8;
    }}

</style>

</head>

<body>

<div class="container">

<h1>Malindu - 20200646</h1>

<div class="sensor-box">Temperature: {temp} °C</div>

<div class="sensor-box">Pressure: {pressure} hPa</div>

</div>

</body>

```

```
</html>
```

```
"""
```

```
return html
```

```
# Starting the Web Server
```

```
addr = socket.getaddrinfo('0.0.0.0', 80)[0][-1]
```

```
s = socket.socket()
```

```
s.bind(addr)
```

```
s.listen(5)
```

```
print("The web server is running...")
```

```
while True:
```

```
    try:
```

```
        conn, addr = s.accept()
```

```
        print("Client is connected from", addr)
```

```
        request = conn.recv(1024)
```

```
        # Reading the sensor data
```

```
        temp = bmp.temperature
```

```
        pressure = bmp.pressure
```

```
        # Sending HTTP Responses
```

```
        response = webpage(temp, pressure)
```

```
        conn.send("HTTP/1.1 200 OK\nContent-Type: text/html\nConnection: close\n\n" + response)
```

```

conn.close()

except Exception as e:

    print("Error:", e)

    conn.close()

```

Appendix C – data_logger

```

import network

import urequests

import time

from machine import Pin, I2C

from bme280 import BMP280


# Setting up WIFI credentials

ssid = "malindu"

password = "malindu123"


# Web App URL for the Google sheet

SCRIPT_URL = "https://script.google.com/macros/s/AKfycbz-istKUmV62nXh8Y6eofC1GSIrHq3bHgEziP-Stg_lageGwpRc8UpaLp1q90yS0aYK/exec"


# Setting up the I2C for BMP280

i2c = I2C(0, scl=Pin(1), sda=Pin(0))

bmp = BMP280(i2c=i2c)

```

```

# Connecting to the WIFI

wlan = network.WLAN(network.STA_IF)

wlan.active(True)

wlan.connect(ssid, password)

print("waiting for the connection...")

while not wlan.isconnected():

    time.sleep(1)

print("Connected Successfully! IP Address:", wlan.ifconfig()[0])


def send_to_sheets(temp, pressure):

    """ Sending sensor readings to Google Sheet """

    try:

        url = f"{SCRIPT_URL}?temp={temp}&pressure={pressure}"

        response = urequests.get(url)

        print("Response:", response.text)

        response.close()

    except Exception as e:

        print("Error sending data:", e)


# Looping of reading and sending data

while True:

    temp = round(bmp.temperature, 2) # Reading the temperature and rounding off for 2 decimals

    pressure = round(bmp.pressure, 2) # Reading the pressure and rounding off for 2 decimals

    send_to_sheets(temp, pressure)

```

```
print(f'Logged: {temp} °C, {pressure} hPa")
time.sleep(10) # Waiting for 10 seconds
```

Appendix D – Apps Script Code

```
function doGet(e) {

    var sheet = SpreadsheetApp.getActiveSpreadsheet().getActiveSheet();

    var timestamp = new Date();

    var temp = e.parameter.temp;
    var pressure = e.parameter.pressure;

    if (!temp || !pressure) {
        return ContentService.createTextOutput("Error: Missing Parameters");
    }

    // Checking if the sheet has headers and if not adding headers
    if (sheet.getLastRow() === 0) {
        sheet.appendRow(["Timestamp", "Temperature (°C)", "Pressure (hPa)"]);
        sheet.getRange("E1").setValue("Total Entries"); // Labelling for total count
    }

    // Appendding the new data
    sheet.appendRow([timestamp, temp, pressure]);
```

```
// Updating the total count in cell  
  
var totalEntries = sheet.getLastRow() - 1;  
  
sheet.getRange("E2").setValue(totalEntries);  
  
    return ContentService.createTextOutput("Data logged successfully. Total Entries: " +  
totalEntries);  
}
```